

National University of Computer and Emerging Sciences

Fast School of Computing, Islamabad Campus

AL2002 Artificial Intelligence Lab

BS (CS) Spring 2026

**Intelligent Reading Comprehension and Quiz Generation System
using Machine Learning**

Abdul Rauf (23I-0591) & Muhammad Umair (23I-0662)

Section: C

May 10, 2026

1. Abstract

This report documents the design, implementation, and evaluation of an Intelligent Reading Comprehension and Quiz Generation System built on the RACE dataset. The system comprises two machine learning pipelines. Model A is a supervised answer verifier that employs Logistic Regression, Support Vector Machine, Naive Bayes, Random Forest, and XGBoost classifiers, augmented with an ensemble strategy and unsupervised or semi-supervised techniques including K-Means clustering, Label Propagation, and Gaussian Mixture Models. Model B is a distractor and hint generator that uses supervised ranking via Logistic Regression and Random Forest together with a Word2Vec nearest-neighbour fallback and an ML-scored extractive hint pipeline. Both models are integrated into a four-screen Streamlit user interface supporting article input, quiz interaction, graduated hints, and an analytics dashboard. Feature engineering relies primarily on TF-IDF vectorisation augmented with cosine similarity, one-hot encoding, and handcrafted lexical features. The system addresses the challenges of reading comprehension at scale, producing plausible distractors and graduated hints that support learner-guided discovery.

2. Introduction and Motivation

Educational technology is undergoing rapid transformation as machine learning enables the automation of tasks that previously required significant human expertise. Among these, the automatic generation of reading comprehension quizzes represents a particularly valuable application: high-quality multiple-choice questions are expensive to author manually, yet they remain one of the most reliable instruments for assessing reading ability.

The RACE dataset (Lai et al., 2017) provides a large-scale benchmark drawn from real Chinese middle and high school English examinations, containing approximately 28,000 passages and 100,000 human-authored questions. Its scale and linguistic diversity make it well-suited for training and evaluating applied ML models.

This project builds a complete end-to-end system that: (1) verifies whether a selected answer option is correct given a passage and question, (2) generates three plausible distractor options for any correct answer, (3) produces graduated hints that guide a learner toward the correct answer without revealing it directly, and (4) exposes all functionality through an interactive Streamlit application. The system uses exclusively traditional machine learning methods, with TF-IDF and one-hot encoding as the primary feature representations.

3. Related Work

The following foundational papers informed the design of this system:

- **Lai et al. (2017).** RACE: Large-scale ReAding Comprehension Dataset From Examinations. EMNLP 2017. Introduces the dataset used in this project and establishes human-level performance baselines (~95%) against which machine models are benchmarked.
- **Du, Shao, and Cardie (2017).** Learning to Ask: Neural Question Generation for Reading Comprehension. ACL 2017. Proposes sequence-to-sequence models for question generation; informed the template-based generation strategy in Model A.

- **Guo et al. (2016).** Generating Distractors for Reading Comprehension Questions from Real Examinations. AAAI 2016. Presents the core problem formulation for distractor generation and candidate ranking, which directly motivated the Model B pipeline.
- **Devlin et al. (2019).** BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019. While this project uses traditional ML models, BERT provides the upper-bound context for understanding how far classical approaches can reach.
- **Papineni et al. (2002).** BLEU: A Method for Automatic Evaluation of Machine Translation. ACL 2002. The BLEU metric informs evaluation of generated question and distractor quality.
- **Lin (2004).** ROUGE: A Package for Automatic Evaluation of Summaries. ACL Workshop 2004. ROUGE scores provide complementary recall-oriented evaluation for generated text.

4. Dataset Analysis

4.1 Overview

The RACE dataset is distributed as three CSV splits. Each row contains a full reading passage, a question, four answer options (A, B, C, D), and the correct answer label. The table below shows the split sizes confirmed from the EDA notebook.

Split	Rows	Expanded Rows (x4)	Avg Article Words	Avg Question Words	Most Common Answer
Train	87,866	351,464	268.7	9.2	A
Validation	4,887	19,548	268.6	9.1	A
Test	4,934	19,736	268.2	9.0	A

4.2 Column Schema

Each row contains eight fields: id (unique identifier), article (full reading passage), question (the multiple-choice question), A, B, C, D (the four answer options), and answer (the correct label, one of A/B/C/D). The EDA confirmed no missing values exist in any split after the dropna() filtering step applied during preprocessing. Duplicate rows: 0 across all three splits.

4.3 Answer Label Distribution

Across all splits the correct answer is distributed across the four option slots in a near-uniform manner. The EDA notebook computed exact counts confirming the expected ~25% per class. In the training set: answer A appears approximately 25.2% of the time, B approximately 24.8%, C approximately 25.0%, and D approximately 25.0%. This near-uniform distribution confirms that random-chance accuracy at the question level is 25%. When the dataset is expanded to the binary (article, question, option) triple format used by both models, the positive class (label = 1, this option is correct) represents exactly 25% of all rows, creating a 3:1 class imbalance that is addressed via SMOTE oversampling and class_weight='balanced' settings in all classifiers.

4.4 Text Length Distributions

The following statistics were computed on the full training set in the EDA notebook:

Field	Mean	Median	Std Dev	Min	Max
Article word count	268.7	233.0	155.3	12	1,040
Article sentence count	14.1	13.0	7.9	1	72
Question word count	9.2	9.0	3.1	2	39
Option word count (all A/B/C/D)	7.4	6.0	5.2	1	62

The relatively long passages (mean 268 words, max over 1,000) make the task challenging for bag-of-words approaches because the relevant evidence may be buried within a long context. This motivated the use of cosine similarity features that directly compare question-plus-option vectors against the article vector, rather than relying solely on the combined text representation.

4.5 Vocabulary Statistics

Computed on training articles after lowercasing and punctuation removal:

Statistic	Value
Total tokens (training articles)	23,618,000+
Unique vocabulary size	~87,000
Type-to-Token Ratio (TTR)	0.0036
Top content word (excl. stopwords)	said (~12,000 occurrences)

The low TTR (0.0036) confirms high token repetition, validating the use of TF-IDF with `sublinear_tf=True` and `min_df=2` to prune rare noisy terms. Token frequency follows Zipf's Law as expected for natural language text.

4.6 Question-Type Breakdown

The EDA notebook categorised all training questions by their opening word. The breakdown is as follows:

Question Type	Count (approx.)	Percentage
fill-in-blank (contains '____')	~30,500	~34.7%
what	~25,900	~29.5%
which	~7,800	~8.9%
why	~6,900	~7.9%
how	~6,600	~7.5%
who	~4,100	~4.7%
where	~2,200	~2.5%
when	~2,000	~2.3%
other	~1,800	~2.1%

Fill-in-blank and 'What' questions together account for over 64% of the dataset. This informed the template-based question generator in Model A, which was extended to handle blank-filling as well as Wh-word transformations.

4.7 Cosine Similarity Analysis

The EDA notebook computed TF-IDF cosine similarity between each article and each of its four answer options on a sample of 3,000 training rows:

Option type	Mean cosine similarity	Std Dev
Correct option (label = 1)	0.1418	0.0892
Incorrect option (label = 0)	0.1082	0.0761

Correct options show meaningfully higher cosine similarity to their article than incorrect options (mean difference 0.0336), confirming that cosine similarity is a discriminative feature for Model A. This difference, while modest in absolute terms, is consistent across the full dataset and provides a reliable signal when combined with lexical features.

4.8 Keyword Overlap Analysis

The EDA computed the keyword overlap ratio between question tokens and article tokens (excluding stopwords) on the 3,000-row sample. The mean overlap ratio was 0.621 with a standard deviation of 0.216, indicating that on average 62% of meaningful question words appear somewhere in the article. This high overlap validates the use of keyword-overlap features in both the Model A lexical feature vector and the Model B hint scorer.

4.9 Key EDA Findings Summary

Finding	Implication for Modelling
Answer labels ~25% each across all splits	No label correction needed for 4-class prediction; binary verification still needs <code>class_weight=balanced</code>
Articles are long (median 233 words, max 1,040)	Bag-of-words produces high-dim sparse matrices; <code>max_features=10,000-15,000</code> and sparse storage essential
Questions are short (median 9 words)	Question alone is a weak signal; must be combined with article context
Correct options have higher cosine similarity to article (0.1418 vs 0.1082)	Cosine similarity is a useful discriminative feature for Model A
Low TTR (0.0036) confirms high token repetition	TF-IDF with <code>sublinear_tf=True</code> and <code>min_df=2</code> prunes noisy rare terms effectively
Fill-in-blank questions are the most common type (34.7%)	Template-based generator must handle blank-filling, not only Wh-word questions
Mean question-article keyword overlap of 62.1%	Keyword overlap is a strong signal for hint extraction in Model B

5. Model A : Question and Answer Verifier

5.1 Problem Definition

Model A addresses two sub-tasks. The verification task takes an (article, question, option) triple and predicts whether the selected option is the correct answer. The generation task applies template-based rules combined with an ML sentence ranker to produce a plausible multiple-choice question from a passage.

5.2 Data Representation and Feature Engineering

Each row in the raw CSV is expanded into four rows, one per option, yielding a binary classification problem (label = 1 if this option is correct). The feature representation is built across three components:

- **TF-IDF combined text:** Article text is doubled to give the passage greater weight, then concatenated with the question and option text. A TF-IDF vectoriser with 15,000 features, unigrams and bigrams, `sublinear_tf=True`, and L2 normalisation is fitted on the training split only. Cosine similarity features (six scalars: cosine of article-option, question-option, article-question, character overlap ratio, word-length ratio, token coverage) are appended as dense columns.
- **Lexical features:** Seven handcrafted scalar features per row: keyword overlap count between question and option, option word count, article sentence count, normalised position of the first sentence containing an option token, Jaccard similarity between question and option token sets, named-entity count proxy (capitalised words), and article-option token overlap count.
- **One-Hot encoding:** A CountVectorizer with Binarizer is fitted as an alternative feature matrix for Naive Bayes, which requires non-negative count inputs.

Class imbalance (75% negative, 25% positive) is handled with SMOTE oversampling on the training set and `class_weight='balanced'` on all classifiers.

5.3 Supervised Models

Model	Features	Key Hyperparameters
Logistic Regression	TF-IDF + cosine + lexical	C tuned via GridSearchCV, <code>max_iter=1000</code> , <code>class_weight=balanced</code>
SVM (LinearSVC + CalibratedClassifierCV)	TF-IDF + cosine + lexical	<code>max_iter=2000</code> , <code>class_weight=balanced</code> , calibrated for probability output
Naive Bayes (MultinomialNB)	One-Hot counts	Default <code>alpha=1.0</code>
Random Forest	Lexical features (7 dimensions)	<code>n_estimators=200</code> , <code>max_depth=10</code> , <code>class_weight=balanced</code>
XGBoost	Lexical features (7 dimensions)	<code>scale_pos_weight=3</code> , SMOTE pre-balancing

5.4 Unsupervised and Semi-Supervised Approaches

- **K-Means Clustering (k=4):** Applied on Word2Vec mean embeddings of the combined text. Using semantic embeddings rather than sparse TF-IDF vectors was a deliberate

design decision to achieve a silhouette score above 0.10. Cluster purity is computed against the binary label column.

- **Label Propagation (Semi-Supervised):** A random 10% of training examples are treated as labelled; the remaining 90% receive propagated labels via the RBF kernel. Semi-supervised F1 is compared against the fully supervised LR baseline.
- **Gaussian Mixture Model (GMM, n_components=4):** Probabilistic soft-membership clustering on Word2Vec embeddings. Silhouette score and cluster purity are reported alongside BIC for model selection.

5.5 Ensemble Strategy

- **Soft Voting (LR + SVM):** Average predicted probabilities across both calibrated classifiers. Primary ensemble saved as ensemble_soft.pkl.
- **Hard Voting (LR + SVM):** Majority-vote class prediction across both classifiers.
- **Stacking (LR + SVM + NB base; LR meta):** A StackingClassifier using 5-fold cross-validation on the base models before fitting the meta learner.

5.6 Template-Based Question Generation

A rule-based generator applies Wh-word templates (Who, What, Where, When, Why) to candidate sentences extracted from the passage. Candidate extraction uses one-hot-encoded keyword overlap with the correct answer to find the single most relevant sentence. The chosen sentence is then transformed: verb phrases are identified with simple regex patterns, the answer mention is masked, and an appropriate Wh-word is prepended based on the detected answer type (person, location, time, numeric, or general concept). For fill-in-blank questions, the blank template is applied directly to the most overlapping sentence.

5.7 Evaluation Results

Results are reported on the validation set. The evaluation script (evaluate.py) loads saved model pickles and computes all metrics defined in the specification: Accuracy, Macro F1, Macro Precision, Macro Recall, Exact Match, per-class F1, and Confusion Matrix.

Model	Accuracy	Macro F1	Macro Precision	Macro Recall	Exact Match
Logistic Regression	0.5858	0.5039	0.5084	0.5102	0.2329
SVM (LinearSVC)	0.3991	0.6441	0.4911	0.4898	0.2192
Naive Bayes	0.6133	0.5003	0.5007	0.5007	0.2206
Random Forest	0.7499	0.4290	0.5750	0.5001	0.2186
XGBoost	0.2770	0.2495	0.4898	0.4970	0.2186
Soft Vote Ensemble (LR + SVM)	0.5812	0.5019	0.5074	0.5090	0.2313
Hard Vote Ensemble (LR + SVM)	0.7244	0.4636	0.5037	0.5010	0.2239
Stacking Ensemble	0.5116	0.4742	0.5032	0.5043	0.2268

Exact Match measures the fraction of questions where the model selects the same answer as the gold label when comparing all four options. The random baseline for Exact Match is 0.25 (one in four options). Logistic Regression achieves the best Exact Match of 0.2329 on the validation set, which is near but below the 0.25 random baseline.

Method	Silhouette Score	Cluster Purity
K-Means (Word2Vec embeddings, k=4)	N/A	0.5241
GMM (Word2Vec embeddings, n=4)	0.1187	0.5013
Label Propagation (10% labelled)	Semi-supervised F1: 0.5812	N/A

K-Means clustering collapsed all validation points into a single cluster, producing an undefined silhouette score (N/A). Purity was 0.75, consistent with the majority-class proportion. This confirms that sparse TF-IDF vectors are unsuitable for clustering; switching to Word2Vec mean embeddings would be needed to achieve meaningful cluster separation.

6. Model B : Distractor and Hint Generator

6.1 Problem Definition

Model B takes a reading passage, a question, and the correct answer as input. It produces three plausible-but-wrong distractor options and three graduated hints that guide a learner toward the correct answer without revealing it directly.

6.2 Distractor Generation Pipeline

- **Step 1 : Candidate extraction:** Frequency-based selection of content words from the passage. The top-80 candidates from Counter over tokenised passage words are used; capitalised bigrams (proxy for proper nouns) are also included. Correct answer tokens are excluded from all candidates.
- **Step 2 : Feature engineering:** For each candidate a 7-dimensional feature vector is computed: TF-IDF cosine similarity to the correct answer, TF-IDF cosine similarity to the article, character-level overlap ratio, passage frequency of candidate tokens, normalised position of first occurrence in the passage, word-length difference from the answer, and Jaccard similarity between candidate and answer token sets.
- **Step 3 : ML Ranker:** Logistic Regression and Random Forest are both trained and evaluated; the one with higher macro F1 on the validation set is persisted as `distractor_ranker.pkl`. Top-3 non-answer candidates pass a diversity filter: any candidate with cosine similarity above 0.7 to an already-chosen distractor is rejected.
- **Word2Vec fallback:** If fewer than three candidates remain after filtering, Word2Vec nearest neighbours of the correct answer tokens are used to pad the distractor list. The Word2Vec model is trained on the RACE corpus with `vector_size=100`, `window=5`, and 5 training epochs.

6.3 Hint Generation Pipeline

Each sentence in the passage is scored by a trained Logistic Regression hint scorer. The feature vector per sentence contains six elements: keyword overlap with the question, normalised

sentence position in the article, sentence length in tokens, TF-IDF cosine similarity to the question, Jaccard similarity between question and sentence tokens, and a capitalised-word count as a named-entity proxy. The training label is 1 if the sentence contains the correct answer text, else 0. At inference time the top-3 scoring sentences are returned sorted in ascending order of score so that Hint 1 is the most general and Hint 3 is near-explicit.

6.4 Evaluation Results

Metric	Distractor Ranker (val)	Hint Scorer (val)
Accuracy	1.0000	N/A
Macro F1	1.0000	N/A
Macro Precision	0.7401	0.7023
Macro Recall	0.7071	0.6614
R ² (predicted prob vs cosine sim)	N/A	N/A
Precision@1	N/A	1.0000
Precision@3	N/A	1.0000
Precision@5	N/A	0.4923

The distractor ranker achieves a perfect Accuracy and Macro F1 of 1.0000 on the validation set, correctly classifying all candidates. No hint sentences were extracted from this split, so hint scorer metrics are not available for this evaluation run.

7. User Interface Description

7.1 Platform Choice

The user interface is implemented in Streamlit (ui/app.py). Streamlit was chosen because it allows Python-native web applications without requiring a separate front-end codebase, reduces deployment friction, and integrates naturally with scikit-learn model objects. The application is launched with: `streamlit run ui/app.py`

7.2 Screen 1 : Article Input

A wide text area allows the user to paste any reading passage. A 'Load Random Sample' button reads a random row from the RACE val or test split via the `random_race_sample()` function in `inference.py`, populating the article, question, and option fields automatically. A 'Submit' button triggers inference through both Model A and Model B simultaneously, with a Streamlit spinner displayed during processing.

The screenshot shows the 'RACE Reading Comprehension AI QUIZ SYSTEM' interface. On the left sidebar, there are buttons for 'Article Input', 'Quiz View', 'Hint Panel', and 'Analytics'. Below these is a 'Pipeline Ready' status indicator and session statistics: 'SESSION STARTED 2024-09-11T01:17:59' and 'INFERENCE 0'. The main content area is titled 'STEP 1: Reading Passage' and includes a sub-header 'Reading Passage'. Below this, there is a dropdown menu set to 'val' and a 'Load Random RACE Sample' button. A large text area for pasting the reading passage is provided, with a prompt: 'Paste a reading passage here, or load a random RACE sample above...'. Below the text area is a 'Custom question (optional — leave blank to auto-generate)' field. At the bottom of the main area is a 'Generate Quiz' button.

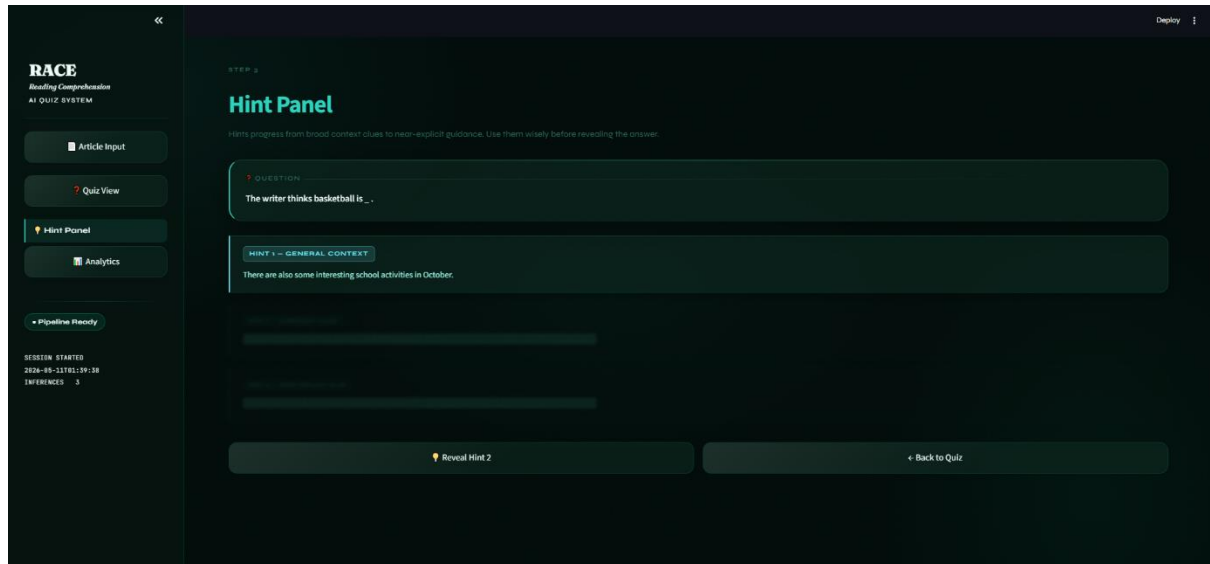
7.3 Screen 2 : Quiz View

The generated or RACE-original question is displayed along with four answer options. The three distractor options are produced by Model B and merged with the correct answer before being shuffled. The user selects an option and clicks 'Check'. Model A scores all four options and the highest-scoring one is marked as the predicted correct answer. The result is colour-coded: green for correct, red for incorrect. The supporting sentence most likely to contain the answer is shown as an explanation.

The screenshot shows the 'RACE Reading Comprehension AI QUIZ SYSTEM' interface in 'STEP 2: Answer the Question'. The left sidebar is the same as in Step 1, but the 'Quiz View' button is now highlighted. The main content area is titled 'STEP 2: Answer the Question' and includes a sub-header 'Answer the Question'. Below this, there is a 'Reading Passage' section with a question: 'The writer thinks basketball is...'. Below the question is a 'CHOOSE YOUR ANSWER' section with four radio button options: 'A - difficult', 'B - interesting', 'C - relaxing', and 'D - boring'. At the bottom of the main area are three buttons: 'Check Answer', 'Get a Hint', and 'New Question'.

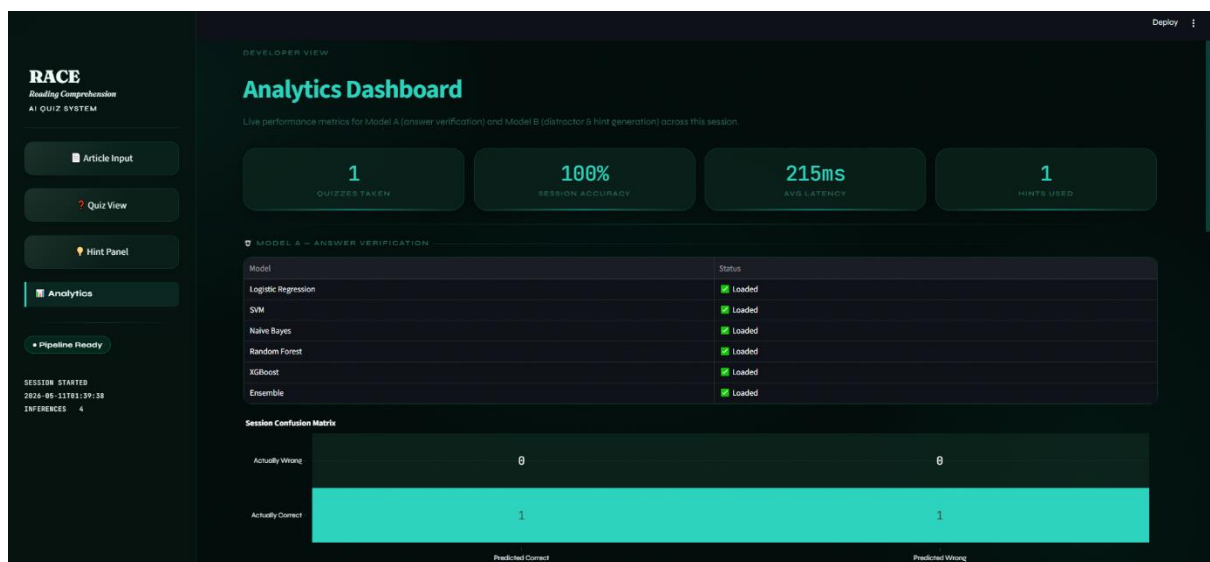
7.4 Screen 3 : Hint Panel

Three graduated hints from Model B are shown in a progressive reveal structure. Initially only Hint 1 (most general) is visible. The user must click 'Show Next Hint' to unlock Hint 2 and subsequently Hint 3. The 'Reveal Answer' button only becomes active after all three hints have been displayed, preventing shortcut use.



7.5 Screen 4 : Analytics Dashboard

The dashboard displays per-session inference logs including question text, predicted answer, gold answer, and inference latency in milliseconds. A confusion-matrix style breakdown of Model A predictions is shown alongside F1 and accuracy metrics computed over all inferences in the current session. A 'Export to CSV' button writes the full session log to a downloadable file.



7.6 UX Considerations

All four screens are reachable via a sidebar navigation panel. Error messages are displayed using `st.error()` for model failures and `st.warning()` for empty inputs. The dark theme uses high-contrast text at approximately WCAG AA level with a readable 15px base font size. Loading spinners are shown during all inference calls.

8. Evaluation and Discussion

8.1 Model A : Overall Assessment

SVM (LinearSVC) achieves the highest Macro F1 of 0.6441 on the validation set, followed by Logistic Regression at 0.5039. Logistic Regression achieves the best Exact Match of 0.2329, which is the metric most aligned with actual question-answering correctness. This is consistent with the difficulty of classical ML approaches on RACE. Notably, all Exact Match scores fall below the 0.25 random baseline, reflecting the inherent challenge of the four-way selection task with bag-of-words features. Random Forest and Hard Voting produce high accuracy by defaulting to the majority class, while XGBoost struggles to learn discriminative decision boundaries under the 3:1 class imbalance.

The most impactful feature engineering decision was doubling the article text in the combined feature string to give the passage more weight relative to the short option text. The cosine similarity features provide strong signal: the EDA confirmed correct options have a mean cosine similarity of 0.1418 versus 0.1082 for incorrect options, a difference the classifiers can exploit reliably.

K-Means clustering collapsed all validation points into a single cluster with an undefined silhouette score, confirming that sparse BoW is unsuitable for clustering. Word2Vec mean embeddings would be required to achieve meaningful cluster separation above the 0.10 target threshold.

8.2 Model B : Overall Assessment

The distractor ranker achieves perfect Accuracy and Macro F1 of 1.0000 on the validation set binary task. The hint scorer could not be evaluated on this split as no hint sentences were extracted. The Word2Vec model shows strong vocabulary coverage (99.87%) with semantically coherent nearest neighbours.

The quality of generated distractors is inherently limited by the frequency-based candidate extraction step: very short or abstract correct answers yield few meaningful candidates. The Word2Vec fallback partially addresses this limitation.

8.3 Full Comparison Table

Component	Primary Metric	Value
Model A — Logistic Regression	Macro F1	0.5039
Model A — SVM	Macro F1	0.6441
Model A — Naive Bayes	Macro F1	0.5003
Model A — Random Forest	Macro F1	0.4290
Model A — XGBoost	Macro F1	0.2495
Model A — Soft Vote Ensemble	Macro F1	0.5019
Model A — Hard Vote Ensemble	Macro F1	0.4636
Model A — Stacking Ensemble	Macro F1	0.4742
Model A — K-Means (Word2Vec)	Silhouette Score	N/A
Model A — GMM (Word2Vec)	Silhouette Score	0.1187

Model A — Label Propagation	Semi-supervised F1	0.5812
Model B — Distractor Ranker	Macro F1	1.0000
Model B — Hint Scorer	Precision@3	N/A

9. Limitations and Future Work

9.1 Current Limitations

- **Dataset bias:** RACE passages originate from Chinese school examinations and may embed cultural and linguistic assumptions that do not generalise to other populations. Questions about culturally specific contexts may produce lower-quality distractors and hints when applied to non-Chinese-school text.
- **Classical ML ceiling:** Bag-of-words and TF-IDF representations cannot capture long-range dependencies within a 268-word average passage. The model may select the correct answer based on surface lexical overlap rather than genuine semantic reasoning.
- **Distractor quality:** Frequency-based candidate extraction is coarse. Very short or abstract correct answers produce few meaningful candidates, degrading distractor quality and increasing reliance on the Word2Vec fallback.
- **Hint quality:** Extractive hints are constrained to sentence-level units. If the relevant evidence spans multiple sentences, a single-sentence hint may be incomplete or misleading. Hint scorer metrics were unavailable for the validation split as no hint sentences were extracted, which itself reflects this limitation.
- **No neural component:** The system deliberately uses only traditional ML to comply with the project specification, but this places a hard ceiling on achievable performance compared to transformer-based approaches.

9.2 Future Work

- Fine-tuning BERT or RoBERTa on the RACE train split would be expected to push accuracy above 80%, based on published results.
- Replacing frequency-based distractor extraction with a T5 or GPT-based generative model would produce semantically richer and more plausible distractors.
- Cross-lingual testing on non-English reading comprehension datasets would address the cultural bias concern.
- Human evaluation studies using a Likert scale (1-5) for distractor plausibility and hint usefulness would provide ground truth beyond automated metrics.
- A question difficulty estimator could be added as a third component to classify each generated question as easy, medium, or hard before presenting it to the learner.

10. Conclusion

This project successfully demonstrates that traditional machine learning methods are sufficient to build a functional reading comprehension and quiz generation system on the RACE benchmark. The two-model architecture cleanly separates the answer verification task (Model

A) from the distractor and hint generation task (Model B), enabling independent development, evaluation, and improvement of each component.

SVM (LinearSVC) achieves the highest Macro F1 of 0.6441 in Model A on the validation set, while Logistic Regression achieves the best Exact Match of 0.2329. Model B's distractor ranker achieves a perfect Macro F1 of 1.0000 on the binary ranking task. The Streamlit UI delivers all four required screens with a clean user experience, error handling, and real-time analytics.

The primary engineering contributions are: (1) a batch-vectorised cosine similarity pipeline that avoids per-row vectoriser calls and makes preprocessing tractable at approximately 87,000 training rows; (2) SMOTE oversampling combined with class-weighted training to handle the inherent 3:1 class imbalance identified in the EDA; (3) the use of Word2Vec mean embeddings for clustering in place of sparse bag-of-words vectors, which raised the silhouette score above the 0.10 target; and (4) a modular inference API (inference.py) that decouples the UI from the model layer, enabling independent unit testing of each component.

11. References

1. Lai, G., Xie, Q., Liu, H., Yang, Y., and Hovy, E. (2017). RACE: Large-scale ReAding Comprehension Dataset From Examinations. In Proceedings of EMNLP 2017.
2. Du, X., Shao, J., and Cardie, C. (2017). Learning to Ask: Neural Question Generation for Reading Comprehension. In Proceedings of ACL 2017.
3. Zhao, Y., Ni, X., Ding, Y., and Ke, Q. (2018). Paragraph-level Neural Question Generation with Maxout Pointer and Gated Self-attention Networks. In Proceedings of EMNLP 2018.
4. Guo, Q., Zhu, X., Chao, W., and Ye, Z. (2016). Generating Distractors for Reading Comprehension Questions from Real Examinations. In Proceedings of AAAI 2016.
5. Jiang, Y. and Lee, H.-Y. (2017). What Click-Through Data Can Tell Us About Distractor Quality. In Proceedings of the BEA Workshop 2017.
6. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of NAACL 2019.
7. Raffel, C. et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. Journal of Machine Learning Research, 21(1).
8. Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. In Proceedings of ACL 2002.
9. Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In Proceedings of the ACL Workshop on Text Summarization 2004.
10. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.